Anemia-Screen — Comprehensive Technical Report

Project: Anemia-Screen · AI Conjunctiva & Palm Anemia Screening and Triage Pipeline
Version: 1.1.0 (app `0.1.0`) · **Date:** 10 October 2026
Platform: Apple MacBook Air (Apple M4, arm64), macOS 27.0 · **Runtime:** conda env `anemia` (Python 3.11.17)
Status: End-to-end working multimodal screening system with two trained models, Clinical NLP symptom intake, dual-site rules synthesis (Eyelid 70% primary / Palm 30% secondary), spatial Grad-CAM explainability, 55/55 automated tests passing, and mandatory clinical disclaimers.

Abstract

Anemia-Screen is a local-first, offline-capable clinical decision-support pipeline that screens for anemia using a dual-modality computer vision and clinical NLP framework. The pipeline ingests smartphone photographs of the **palpebral conjunctiva** (inner lower eyelid — primary site, 70% weight) and/or the **palm** (palmar skin — secondary confirmatory site, 30% weight), performs rigorous data-calibrated image quality gating, runs specialized MobileNetV3 classifiers, generates true spatial Grad-CAM explainability heatmaps, and synthesizes vision predictions with a WHO/MoHFW-aligned clinical rules engine and free-text symptom NLP parser. The system strictly adheres to the safety-first principle:
"An image is a screening tool, NOT a diagnosis" — non-invasive visual screening never initiates pharmacological treatment on its own; it triages patients for timely confirmatory laboratory hemoglobin testing.

Two models were trained on **three real public datasets** (Eyes-defy-anemia India/Italy, CP-AnemiC Ghana, and a palmar images dataset — 5,172 images total). Through the true serving-preprocessing path the conjunctiva model achieves **80.9% sensitivity / 60.0% specificity**, and the palm model **76.5% sensitivity / 69.7% specificity** on patient-wise validation splits. The system provides an interactive dual-modality launcher (`run_pipeline.py`) featuring crash-free native file pickers, clinical symptom NLP with negation handling ("denies breathlessness"), automatic WHO danger-sign escalation, standalone Grad-CAM heatmap exports, a 2-row × 3-column master visual dashboard (`multimodal_triage_dashboard.png`), standalone 3-panel figures for each site, and a standardized clinical disclaimer embedded in all JSON reports, figures, and CLI outputs. The codebase is supported by **55 passing automated tests**, a self-contained offline FastAPI service, web UI, and batch CSV processor.

Table of Contents

1. Problem Statement & Clinical Background
2. System Architecture
3. Environment & Technology Stack
4. Datasets
5. Data Engineering (Manifests, Splits, Anti-Leakage)
6. Image I/O (HEIC/iPhone Support)
7. Image Quality Gate
8. Preprocessing Pipeline
9. Models & Training
10. Training Results
11. Honest Serving-Path Evaluation (Sensitivity / Specificity / Confusion Matrices)
12. Explainability (Grad-CAM)

13. WHO/MoHFW Clinical Rules Engine
14. Interfaces (REST API, Web UI, CLI, Batch)
15. Software Quality & Testing
16. Engineering Issues Found & Fixed During the Build
17. Version History (Git)
18. Known Limitations & Honest Caveats
19. Roadmap
20. Runbook (How to Run Everything)
21. References

1. Problem Statement & Clinical Background

1.1 The problem

Anemia affects roughly half of pregnant women (~52%) and about two-thirds of children 6–59 months (~67%) in India (NFHS-5, 2019–21). Detection is late because confirmation needs a blood sample and a device that sub-centres often lack; the result is late treatment and unclosed follow-up loops.

1.2 The approach

Conjunctival (inner lower-eyelid) pallor is the classical clinical sign for anemia assessment (WHO pallor assessment sites: conjunctiva, palms, nail beds). The conjunctiva has a dense, exposed capillary bed with no overlying pigmented skin, so its redness tracks hemoglobin comparatively well across skin tones. This project converts that clinical sign into an automated pipeline:

```
photo + patient JSON
   → Quality Gate (reject bad photos before inference)
   → Preprocessing (modality-aware crop + colour normalization)
   → MobileNetV3 inference (per-modality trained model)
   → Grad-CAM explainability heatmap
   → WHO/MoHFW rules engine (safety-first triage)
   → structured JSON triage report (+ CSV for batch dashboards)
```

1.3 Clinical framing (plan §4, Anemia Loop spec §7.3)

The task is framed as **3-class classification, not Hb regression** — predicting an exact hemoglobin value from a small image dataset would overstate the system's capability. The WHO-aligned classes are:

Class	Hemoglobin (g/dL)
Normal	Hb ≥ 11.0
Mild/Moderate	7.0 – 10.9
Severe	Hb < 7.0

The vision model additionally exposes a continuous **P(anemic)** = 1 − P(Normal), which the rules engine compares against configurable risk thresholds.

1.4 Positioning

The pipeline plugs into the **Anemia Mukt Bharat (AMB)** test-and-treat workflow — it does not replace it. It counsels and reminds only; dosing always follows the ANM/Medical Officer and MoHFW protocol. It is a **screening and triage aid, NOT a diagnostic device**; every abnormal result must be confirmed with a measured hemoglobin.

2. System Architecture

2.1 Pipeline flow (Dual-Modality & Clinical NLP Architecture)

```
INPUT: Eyelid image + Palm image + Patient Clinical Factors (Age, Sex, Pregnancy, Hb, Symptoms
    Text)
    │
    ├── [0] CLINICAL NLP INTAKE (app/clinical_nlp.py)
    │       → Lexicon normalization & negation parsing ("denies breathlessness", "no chest pain")
    │       → WHO/MoHFW danger-sign detection (fainting, chest pain, palpitations, severe pallor…)
    │       → Clinical entity extraction: Pregnancy status, Trimester, Measured Hb (g/dL)
    │       → Pre-eclampsia risk detection (swelling + headache/blurred vision)
    │
    ├── [1] IMAGE I/O (app/imageio.py)
    │       → OpenCV with Pillow / pillow-heif fallback, EXIF orientation correction
    │
    ├── [2] QUALITY GATE (app/pipeline/quality_gate.py)
    │       → Modality-calibrated blur (Laplacian variance: 50 for Eye, 3 for Palm)
    │       → Brightness window, exposure clipping, specular glare & conjunctiva tissue checks
    │       → Fail → RETAKE_PHOTO recommendation (prevents false predictions)
    │
    ├── [3] PREPROCESSING (app/pipeline/preprocessing.py)
    │       → Eye: HSV-red conjunctiva auto-crop (with zero-size safety guards)
    │       → Palm: Center crop 85%
    │       → Both: Gray-world white balance + CLAHE local contrast → 224² resize → ImageNet
       normalization
    │
    ├── [4] INFERENCE (app/pipeline/inference.py)
    │       → timm MobileNetV3-large-100 per modality → RiskClass, confidence, P(anemic), class
       logits
    │
    ├── [5] EXPLAINABILITY (app/pipeline/gradcam.py)
    │       → Pre-pooling conv layer (model.blocks[-1]) for true 2D spatial feature activation maps
    │       → Standalone JET heatmap + 45/55 overlay on normalized ROI
    │       → Saved to disk (eyelid_heatmap.png, palm_heatmap.png) & embedded in JSON (base64 PNG)
    │
    └── [6] WHO/MoHFW MULTIMODAL RULES SYNTHESIS (app/pipeline/rules_engine.py)
            → Danger-sign override (SAFETY FIRST: immediate REFER_NOW)
            → Measured-Hb override (confirmatory blood test priority: <7 REFER_NOW; 7–10.9
                TREAT_AND_RECHECK; ≥11 ROUTINE)
            → Dual-site synthesis: Eyelid (70% primary weight) + Palm (30% secondary weight)
            → Site concordance / discordance analysis & safety escalation
            → Quality failure fallbacks (e.g., eyelid failed → fallback to palm with audit flag)
    │
OUTPUT:
   • Master Multimodal Dashboard (outputs/multimodal_triage_dashboard.png, 2-row × 3-col panel)
   • Isolated 3-Panel Figures (outputs/eyelid_3panel.png, outputs/palm_3panel.png)
   • Standalone Heatmaps (outputs/eyelid_heatmap.png, outputs/palm_heatmap.png)
   • Full JSON Triage Report (outputs/multimodal_triage_report.json) + CSV Batch Dashboards
   • Color-coded Terminal Report ending with the MANDATORY CLINICAL DISCLAIMER
```

2.2 Modality design & Dual-Site Synthesis

The system supports both **isolated single-site screening** and **integrated dual-modality screening**:

Feature	`eye` (Primary Factor — 70% Weight)	`palm` (Secondary Factor — 30% Weight)
Biological Target	Palpebral conjunctiva (inner lower eyelid capillary bed)	Palmar skin (palmar pallor) / nail beds
Physiological Rationale	Direct microvascular visualization; minimal melanin interference	Peripheral cutaneous vasoconstriction; easy accessibility
Model Checkpoint	`ml/checkpoints/anemia_eye_mobilenetv3.pt`	`ml/checkpoints/anemia_palm_mobilenetv3.pt`
Serving Preprocessing	HSV-red conjunctiva auto-crop + CLAHE + Gray-World	Center crop 85% + CLAHE + Gray-World

Feature	`eye` (Primary Factor — 70% Weight)	`palm` (Secondary Factor — 30% Weight)
Blur Threshold (Laplacian)	50 (sharp vascular edges required)	3 (smooth cutaneous tissue naturally low-edge)
Tissue & Glare Gates	Active (red tissue fraction ≥ 0.05, glare fraction ≤ 0.15)	Cutaneous skin check; glare/tissue checks skipped
Validation Accuracy	60.99%	77.75%
Serving Sensitivity	80.9%	76.5%
Serving Specificity	60.0%	69.7%
Multimodal Role	**Primary biomarker (70% fused probability weight)**	**Corroborating biomarker (30% fused probability weight)**

Dual-Site Decision Synthesis:

- When both images pass the quality gate:

$$P_{anemic}^{fused} = 0.70 \times P_{anemic}^{eyelid} + 0.30 \times P_{anemic}^{palm}$$

- Site agreement is audited: `concordant` if both models predict the same risk class; `discordant` if they differ. When discordant (e.g. eyelid Normal but palm Severe), the system flags `discordant_screening_sites` and escalates caution without overriding high-risk indications.
- If the eyelid photo fails quality gate (e.g., motion blur), the engine falls back to palm with the flag `eyelid_quality_failed_fallback_palm`.

2.3 Component map

Component	File(s)	Responsibility
Pipeline Launcher	`run_pipeline.py`	Top-level interactive runner invoking UI selector and pipeline
Clinical Intake UI	`scripts/ui_selector.py`	Native macOS AppleScript individual image pickers + patient factors intake dialog + CLI fallback
Clinical NLP Analyzer	`app/clinical_nlp.py`	Free-text symptom lexicon parsing, negation detection, danger-sign escalation, entity extraction
FastAPI service	`app/main.py`	`/analyze`, `/health`, `/api`, offline `/docs`, web UI at `/`
Runtime config	`app/config.py`	Model paths, input size, quality-gate thresholds (env-overridable `ANEMIA_*`)
Clinical config	`app/clinical_config.yaml`, `app/clinical.py`	WHO Hb bands per group, risk thresholds, danger signs, follow-up windows
Schemas	`app/schemas.py`	Pydantic contracts (`PatientMetadata`, `QualityReport`, `VisionPrediction`, `TriageDecision`, `TriageReport`, `CLINICAL_DISCLAIMER`)
Image I/O	`app/imageio.py`	HEIC/HEIF + EXIF-aware robust image loader
Quality gate	`app/pipeline/quality_gate.py`	Modality-calibrated blur / brightness / exposure / glare / tissue checks
Preprocessing	`app/pipeline/preprocessing.py`	Modality-aware crop + dimension guards + colour normalization + tensor conversion
Inference	`app/pipeline/inference.py`	Per-modality MobileNetV3 classifier + graceful fallback
Grad-CAM	`app/pipeline/gradcam.py`	Pre-pooling conv feature map targeting, standalone heatmap & overlay generation
Rules engine	`app/pipeline/rules_engine.py`	Pure WHO/MoHFW triage logic + dual-modality 70/30 synthesis (`generate_multimodal_triage_decision`)
Orchestrator	`app/pipeline/orchestrator.py`	Wires stages 1–6 per modality, returns structured `TriageReport`
Manifest builder	`ml/build_manifest.py`	Parses 3 raw datasets → `data/manifests/{eye,palm}.csv`
Dataset	`ml/dataset.py`	Serving-consistent dataset + patient-wise train/val splits
Training	`ml/train.py`	Per-modality MobileNetV3 training loop with class weighting
Evaluation	`ml/evaluate.py`	Honest serving-path sensitivity / specificity evaluation
ONNX export	`ml/export_onnx.py`	Checkpoint → ONNX model (verified with onnxruntime)
CLI analyzer	`scripts/analyze_cli.py`	Multimodal & single-image CLI, master 2×3 dashboard, standalone heatmaps, terminal report
Batch processor	`scripts/batch_process.py`	Folder of images → triage CSV (API or local mode)
Sample generator	`scripts/make_sample_images.py`	Synthetic demo images + metadata sidecars
Web UI	`app/static/index.html`, `app/static/app.js`	Offline, self-contained single-page upload & triage application
Test suite	`tests/`	**55 automated tests** across 8 test modules

3. Environment & Technology Stack

Layer	Choice	Installed version
OS / hardware	macOS 27.0, Apple M4 (arm64), **MPS** GPU acceleration	—
Python	conda env `anemia` (isolated from base)	3.11.17
Deep learning	PyTorch + torchvision	2.14.1 / 0.29.1
Model zoo	`timm` — `mobilenetv3_large_100`	1.0.30
Serving	FastAPI + uvicorn[standard]	0.143.0 / 0.54.0
Validation	pydantic + pydantic-settings	2.14.0 / 2.15.0
Imaging	OpenCV, Pillow, **pillow-heif** (HEIC/iPhone)	4.10.0.84 / 12.3.0 / 1.8.0
Explainability	grad-cam (`pytorch_grad_cam`)	1.5.7
Portability	onnx + onnxruntime + onnxscript	1.23.2 / 1.31.0 / 0.7.2
Data	pandas, numpy, openpyxl	2.2.3 / 2.1.3 / 3.1.5
Notebooks	jupyterlab + ipykernel (`anemia` kernel)	4.6.4 / 7.4.0
Testing	pytest + httpx	9.1.1 / 0.28.1
Archive	rarfile + patool + `unar` (homebrew)	CP-AnemiC `.rar` extraction
Config	PyYAML	6.0.3

Pinned in `requirements.txt` / `environment.yml`. **No paid APIs; fully local/offline.**

4. Datasets

Three real public datasets (auto-detected in the project root by `ml/build_manifest.py`):

4.1 Eye modality — 912 images

Source	Images	Normal	Mild/Moderate	Severe	Label source
Eyes-defy-anemia — India	95	55	40	0	`India.xlsx` (`Number`, `Hgb`, `Gender`, `Age`)
Eyes-defy-anemia — Italy	107	93	14	0	`Italy.xlsx` (same schema)
CP-AnemiC — Ghana	710	286	376	48	`Anemia_Data_Collection_Sheet.xlsx` (`IMAGE_ID`, `HB_LEVEL`, `Severity`, `Age(Months)`, `GENDER`)
Total (eye)	**912**	**434**	**430**	**48**	real measured Hb → WHO 3-class mapping

Notes:

- Eyes-defy-anemia stores per-subject folders (`India/61/`, `Italy/30/`) each containing the original `.jpg` plus segmentation masks (`_palpebral`, `_forniceal`, `_forniceal_palpebral`). The **original jpg** is used as model input; conjunctiva cropping happens at preprocessing time.
- CP-AnemiC ships as a `.rar` (extracted with `unar` during setup). Images are clinical conjunctiva crops, often on **black backgrounds** — a fact that drove quality-gate recalibration (§7.3). Age metadata (6–59 months cohort) is available for child triage.

4.2 Palm modality — 4,260 images

Source	Images	Anemic	Non-anemic	Label source
Palmar images dataset (`Palm/`)	4,260	2,562	1,698	binary label in filename (`Anemic*` / `Non-Anemic*`)

- Labels are **binary** (no Hb values) — anemic maps to the conservative proxy class `Mild/Moderate`, non-anemic to `Normal`.

- Verified programmatically (ASCII colour-map rendering of samples) that these are **palm photos** — full-frame uniform pinkish skin, no pale nail plates — matching the known "Detecting Anemia Based on Palm Images" dataset family (`AnemicP-NNN` naming, P = palmar).

4.3 WHO 3-class mapping (from measured Hb)

`hb < 7.0 → Severe` · `7.0 ≤ hb < 11.0 → Mild/Moderate` · `hb ≥ 11.0 → Normal`

5. Data Engineering (Manifests, Splits, Anti-Leakage)

5.1 Manifest builder (`ml/build_manifest.py`)

One command parses all three raw dataset formats into unified per-modality manifests:

```
data/manifests/eye.csv   columns: image_path, hb, label, patient_id, source, modality
data/manifests/palm.csv  columns: image_path, hb (empty), label, patient_id, source, modality
```

Parsers per dataset:

- **Eyes-defy-anemia**: reads `Number`/`Hgb` from each country `.xlsx`; resolves each subject folder; picks the original `.jpg` (never the `_palpebral`/`_forniceal` mask PNGs); patient id = `eyesdefy_{country}_{Number}`.
- **CP-AnemiC**: reads `IMAGE_ID`/`HB_LEVEL` from `Anemia_Data_Collection_Sheet.xlsx` (case-insensitive key match); real Hb → WHO band; patient id = `cp_anemic_Image_NNN`.
- **Palm**: regex on filename (`non-anemic*` → Normal; `anemic*` → Mild/Moderate proxy); strips `(n)` duplicate markers for pseudo-patient ids.

5.2 Anti-leakage: patient-wise splits

`patient_wise_split()` (in `ml/dataset.py`) splits **by patient id**, never by image: the patient list is shuffled with a fixed seed (42), ~20% of *patients* go to validation, and every image of a given patient lands in exactly one split. This prevents the classic medical-ML leakage where the same patient's photos appear in both train and validation.

5.3 Train/serve consistency (a critical design decision)

Training uses the **exact serving preprocessing** (`app.pipeline.preprocessing.preprocess`: modality-aware crop → gray-world + CLAHE → 224² → ImageNet normalize) for every sample; training-only augmentation (flip, ±15° rotation, brightness/contrast jitter, mild defocus) is applied **before** that pipeline. This guarantees the model sees the identical input distribution at train and serve time. (This decision came from a real failure — see §16.1.)

6. Image I/O (`app/imageio.py`)

- Primary decoder: OpenCV `imdecode` (fast path for jpg/png/bmp/webp).
- Fallback: Pillow (with **pillow-heif** registered) — decodes **HEIC/HEIF (iPhone photos)** and any other format OpenCV misses; applies **EXIF transpose** so sideways phone photos are upright.
- `read_image(path)` / `decode_image(bytes)` both return OpenCV-style **BGR uint8** arrays.
- All entry points (API upload, CLI, batch, dataset loader) use this module.

7. Image Quality Gate

7.1 Checks (per image, before any inference)

Check	Metric	Threshold (eye)	Threshold (palm)	Failure reason
Focus	variance of Laplacian	≥ 50	≥ 3	"Image too blurry"
Brightness	mean luminance (0–255)	15–215	15–215	"too dark" / "too bright"
Exposure clipping	fraction of pixels ≤5 or ≥250	≤ 85%	≤ 85%	"Excessive clipping"
Glare	fraction of very bright + desaturated pixels (HSV V≥245, S≤30)	≤ 8%	≤ 8%	"Too much glare"
Conjunctiva tissue	reddish HSV fraction (two hue bands 0–12 & 168–179, S≥40, V≥40)	≥ 2%	skipped	"Little conjunctiva tissue"

Failed images get `RETAKE_PHOTO` with the specific reasons — **no inference is performed** (prevents garbage-in false positives).

7.2 Resolution normalization

Every image is downscaled so its longest edge = **1024 px** before metrics are computed. This makes blur/exposure measurements **resolution-independent** (a sharp 12-MP iPhone photo would otherwise score artificially low Laplacian variance: measured focus 5.8 at 4032×3024 vs 204.8 after normalization — same photo).

7.3 Data-calibrated thresholds (methodology)

Initial thresholds (tuned on synthetic phone-like images) **rejected 100% of the real eye dataset**. Thresholds were then calibrated by measuring the actual distribution across the full manifest (n=912 eye, n=4,260 palm):

Metric	eye dataset p5 / median	palm dataset p5 / median	Final threshold
blur variance	21.7 / 1513.7	3.6 / 22.6	eye 50 · palm 3
brightness	17.5 / 43.3	90.0 / 146.1	min 15
under-exposed fraction	up to ~0.8 (black clinical backgrounds)	—	clip ≤ 0.85

The CP-AnemiC clinical crops sit on black backgrounds (up to ~80% near-black pixels by design), which is legitimate for that data — hence the raised clip tolerance while a fully black frame still fails.

8. Preprocessing (`app/pipeline/preprocessing.py`)

1. **Crop (modality-aware):**
   - `eye`: HSV red-mask (two hue bands: 0–12 and 168–179) → morphological open/close → largest contour → bounding box + 8% padding. Falls back to a 70% centre crop if no valid red region is found.
   - `palm`: centre crop (85%).
   - **Dimension Safety Guards:** Both cropping pathways include explicit defensive checks (`crop.size > 0 and crop.shape[0] > 0 and crop.shape[1] > 0`) to guard against zero-pixel bounding slices or degenerate contours, eliminating OpenCV resize assertion crashes.
2. **Colour normalization:** gray-world white balance (per-channel mean equalisation) + CLAHE (L channel in LAB space, clip limit 2.0, 8×8 tile grid) — reduces illumination, skin tone, and camera sensor variance.
3. **Tensor conversion:** resize 224×224 → RGB → /255 → ImageNet normalization (mean `[0.485, 0.456, 0.406]`, std `[0.229, 0.224, 0.225]`) → CHW float32.

The same function serves training, validation, evaluation, and live inference (§5.3).

9. Models & Training

9.1 Architecture

- **Backbone:** `mobilenetv3_large_100` via `timm`, ImageNet-pretrained, **new 3-class head** (Normal / Mild/Moderate / Severe — fixed order).
- One model instance per modality (`AnemiaClassifier(modality=...)`, lazily cached per modality; MPS → CUDA → CPU device resolution).
- **Graceful fallback:** if no checkpoint exists (or loading fails), the classifier runs ImageNet-pretrained weights with a fresh head and flags every prediction `model_trained=false` — the pipeline never hard-fails for a missing model.

9.2 Training configuration

Parameter	eye	palm
Epochs	12	8
Batch size	32	64
Optimizer	AdamW (lr 3e-4, weight decay 1e-4)	AdamW (lr 3e-4, weight decay 1e-4)
LR schedule	CosineAnnealingLR	CosineAnnealingLR
Loss	CrossEntropy, **class-weighted** (inverse-frequency)	CrossEntropy, class-weighted
Split	patient-wise 80/20, seed 42	patient-wise 80/20, seed 42
Augmentation	flip 50% · rotate ±15° 50% · brightness/contrast ±20% 50% · defocus 20% (applied **before** the serving preprocessing)	same
Device	MPS (Apple M4)	MPS (Apple M4)
Checkpointing	best validation accuracy saved each epoch (`{model, arch, classes, modality, val_acc, epoch}`)	same

10. Training Results

10.1 Eye model (conjunctiva, 912 images) — final training run

Epoch	train loss	val loss	val acc
1	3.596	2.418	0.440
2	1.746	2.174	0.489
3	1.500	2.301	0.593
4	1.109	2.048	0.582
5	1.078	1.776	0.571
6	0.914	1.998	0.533
7	0.774	1.837	0.566
8	0.590	1.735	**0.610** ← best (checkpoint)
9	0.548	1.713	0.599
10	0.482	1.696	0.582
11	0.749*	1.124*	0.550*
12	0.800*	1.116*	0.550*

*Epochs 11–12 from the earlier (pre-train/serve-consistency) run for reference; the saved checkpoint is epoch 8 at **val_acc 0.6099 (60.99%)**.

10.2 Palm model (4,260 images) — final training run

Epoch	train loss	val loss	val acc
1	0.927	1.065	0.702
2	0.274	0.900	0.765
3	0.139	1.012	**0.778** ← best (checkpoint)
4	0.102	1.367	0.729
5	0.066	1.016	0.757
6	0.036	1.117	0.749
7	0.037	1.162	0.740
8	0.036	1.053	0.760

Saved checkpoint: epoch 3 at **val_acc 0.7775 (77.75%)**. (Train loss ≪ val loss indicates overfitting from epoch ~3 onward — early best-checkpoint selection mitigates it; more data or stronger regularization would help.)

11. Honest Serving-Path Evaluation (`ml/evaluate.py`)

Method: run the trained checkpoint over a **random representative sample of the patient-wise validation split**, through the **exact serving path** (image I/O → quality gate → preprocessing → inference — no training shortcuts). Anemic-positive = Mild/Moderate + Severe.

11.1 Eye model (150 val images, 37 rejected by the quality gate — reported honestly)

Confusion matrix (rows = true, cols = predicted; order Normal / Mild-Moderate / Severe):

```
            Normal  M/M   Severe
Normal          27    15       3
M/M             12    37       9
Severe           1     7       2
```

- **Sensitivity (anemic): 80.9%** (55/68)
- **Specificity (normal): 60.0%** (27/45)
- Sensitivity-first is the correct clinical priority: missing an anemic patient is worse than a false alarm, and every positive is confirmed by blood test anyway.

11.2 Palm model (200 val images, 2 rejected)

```
            Normal  M/M   Severe
Normal          46    20       0
M/M             31   101       0
Severe           0     0       0
```

- **Sensitivity (anemic): 76.5%** (101/132)
- **Specificity (normal): 69.7%** (46/66)

11.3 Interpretation & honest caveats

- The eye task is genuinely harder: 3 countries, different cameras, 3-class Hb-derived labels, and 48-image severe class. The palm dataset is a cleaner, near-binary task — hence the gap.
- Confusion concentrates between **adjacent** classes (Normal ↔ Mild/Moderate; M/M ↔ Severe), not catastrophic cross-class errors — appropriate for a triage tool that escalates borderline cases to confirmatory testing rather than treating from the image.
- All numbers are **validation** metrics; a final untouched test set was not carved out (noted as a limitation in §19).

12. Explainability (Grad-CAM)

12.1 Target Layer Architecture & The 1×1 Spatial Resolution Fix

In convolutional networks, Grad-CAM requires a feature map retaining two-dimensional spatial dimensions (H × W). In `timm`'s `mobilenetv3_large_100` architecture, the network layout is:

```
model.blocks (inverted residual blocks) → model.conv_head → model.global_pool → model.classifier
```

An initial naive search for the last `Conv2d` in `model.modules()` returned `conv_head`. However, `conv_head` sits immediately before/at the pooling stage where spatial variance collapses. In earlier runs, backpropagating gradients into `conv_head` produced identical spatial activations across all coordinates (`gcam min: 0.0, max: 0.0`), rendering a flat, uninformative blue heatmap.

The Fix: `_find_target_layer` in `app/pipeline/gradcam.py` was rewritten to explicitly target `model.blocks[-1]` — the final depthwise/pointwise convolutional block *before* spatial pooling. This restored true 224 × 224 spatial feature activation maps (`min: 0.0, max: 1.0, std: 0.28`), accurately highlighting the vascular bed in eyelid images and cutaneous capillary zones in palm images.

12.2 Rich Artifact Pipeline (`generate_gradcam_artifacts`)

The Grad-CAM subsystem generates multiple distinct artifacts:

1. **Overlay Image (`overlay_bgr` / `overlay_png_b64`):** Normalized ROI blended 55% base image + 45% JET colormap.
2. **Pure Heatmap (`heatmap_bgr` / `heatmap_png_b64`):** Standalone color-mapped activation map (JET spectrum) without base image blending.
3. **Raw Activation Array (`cam_raw`):** 2D normalized float array [0.0, 1.0] for quantitative analysis.
4. **Standalone Disk Exports:** Exported automatically by the CLI to `outputs/eyelid_heatmap.png` and `outputs/palm_heatmap.png`.
5. **Standalone 3-Panel Figures:** Exported to `outputs/eyelid_3panel.png` and `outputs/palm_3panel.png`.

Robustness guarantees: Automatic **CPU fallback** if the Apple Silicon MPS backend lacks a specialized backward kernel; graceful `None` fallback with warnings if `pytorch_grad_cam` is unavailable.

13. WHO/MoHFW Clinical Rules Engine

13.1 Safety design principles

1. **SAFETY FIRST** — rules evaluated most-urgent-first; a dangerous presentation can never be downgraded by a milder rule.
2. **IMAGE IS A SCREEN, NOT A DIAGNOSIS** — the vision score alone never triggers treatment or referral; it triggers a *confirmatory Hb test* (`CONFIRM_48H`). Only a measured Hb < 7 or a danger sign produces `REFER_NOW`.
3. **CONFIG-DRIVEN** — every threshold lives in `app/clinical_config.yaml`; WHO-2011 → WHO-2024 trimester-specific cut-offs (or MoHFW updates) can be switched without code changes.

13.2 Decision table (priority order)

#	Condition	Action	Urgency
1	Danger sign (breathlessness, fainting, chest pain, severe pallor, palpitations, confusion, syncope…)	`REFER_NOW`	High
2	Measured Hb < 7 g/dL (group-specific cut-offs)	`REFER_NOW`	High
3	Measured Hb 7.0–10.9 (mild/moderate)	`TREAT_AND_RECHECK` (IFA per MoHFW; re-check 4–8 weeks)	Medium
4	Measured Hb ≥ 11 (normal) — **overrides the image**	`ROUTINE`	Low
5	High image risk (P(anemic) ≥ 0.60)	`CONFIRM_48H` (blood test within 48h)	High
6	Medium risk (P ≥ 0.30) or ≥ 2 symptoms	`CONFIRM_1WEEK`	Medium
7	Low risk, no symptoms	`ROUTINE` (IFA, re-screen at next visit)	Low
—	Quality gate failure	`RETAKE_PHOTO`	None

Pre-eclampsia flag: pregnant + (swelling + headache/blurred vision) → `preeclampsia_check` flag appended (BP check) per the pregnancy protocol.

13.3 Group-specific WHO Hb bands (4-level, from `clinical_config.yaml`)

Band	Pregnant	Child 6–59 mo	Default
Normal	≥ 11.0	≥ 11.0	≥ 11.0
Mild	10.0–10.9	10.0–10.9	10.0–10.9
Moderate	7.0–9.9	7.0–9.9	7.0–9.9
Severe	< 7.0	< 7.0	< 7.0

Group resolution: pregnant → `pregnant`; age 6–59 months → `child_6_59m`; else `default`.

13.4 TriageDecision output

`action_code` (machine-readable enum: `REFER_NOW` / `CONFIRM_48H` / `CONFIRM_1WEEK` / `TREAT_AND_RECHECK` / `ROUTINE` / `RETAKE_PHOTO`), `action` (human instruction), `urgency`, `rationale` (explainable clinical reasoning string), `rule_fired`, `hb_band`, `recheck_in_weeks`, `flags`. The engine is pure/deterministic — unit tests pin its behaviour, including override priorities and boundary Hb values.

13.5 Dual-Modality Decision Synthesis (`generate_multimodal_triage_decision`)

In dual-site screening, the engine synthesizes findings from both the **Eyelid** (primary factor) and **Palm** (secondary factor):

1. **Weighted Probability Fusion:**

$$P_{anemic}^{fused} = 0.70 \times P_{anemic}^{eyelid} + 0.30 \times P_{anemic}^{palm}$$

   Eyelid conjunctival pallor receives 70% weight due to direct microvascular visualization with zero overlying melanin. Palm cutaneous pallor provides 30% confirmatory weight.

2. **Clinical Class Fusion:**
   - If Eyelid predicts `Severe` → Fused risk is `Severe`.
   - If Eyelid predicts `Mild/Moderate` → Fused risk is `Mild/Moderate` (escalated to `Severe` only if Palm is `Severe` AND P(fused) ≥ 0.75).
   - If Eyelid predicts `Normal` → Fused risk is `Normal`, unless Palm is `Severe` or P(fused) ≥ 0.60 (in which case it escalates to `Mild/Moderate` for screening safety).
3. **Concordance Audit:**
   - `concordant` if both models assign identical risk classes.
   - `discordant` if classes differ; appends `discordant_screening_sites` flag and clinical explanatory rationale.
4. **Single-Site Fallback:**

- If Eyelid fails the quality gate but Palm passes, Palm serves as secondary fallback with the audit flag `eyelid_quality_failed_fallback_palm`.

13.6 Clinical NLP Symptom Analyzer (`app/clinical_nlp.py`)

A dedicated natural language processor ingests unstructured symptom notes recorded by frontline health workers:

- **Lexicon Matching:** Matches canonical symptoms against a dictionary of clinical and vernacular terms across 18 symptom categories.
- **Negation Handling:** Uses regex lookback analysis (~25 characters) to identify negation prefixes (`no`, `not`, `denies`, `without`, `never`, `free of`, `negative for`). For example, "patient feels fatigue, denies breathlessness, no chest pain" extracts `fatigue` while correctly discarding `breathlessness` and `chest pain`. Blanket negations ("none", "nil", "asymptomatic", "healthy") are handled explicitly.
- **Danger Sign Escalation:** Detects WHO/MoHFW red flags (`breathlessness`, `fainting`/`syncope`, `chest pain`, `severe pallor`, `palpitations`, `confusion`, `unable to walk`) and escalates clinical urgency to `High` / `REFER_NOW`.
- **Entity Extraction:** Automatically detects pregnancy mentions, trimester (`1st`/`2nd`/`3rd`), and recorded hemoglobin values (e.g., "hb 9.2 g/dl").
- **Pre-eclampsia Risk Rule:** Identifies co-occurring peripheral edema (`swelling`) with neurological symptoms (`headache`, `blurred vision`) in pregnant patients and sets `preeclampsia_check`.

13.7 Mandatory Clinical Disclaimer

All system interfaces, reports, and exported figures mandate a prominent clinical disclaimer (`CLINICAL_DISCLAIMER` in `app/schemas.py`):

> **DISCLAIMER:** This screening tool is an AI-assisted risk assessment and is NOT a medical diagnosis. Non-invasive pallor screening cannot replace laboratory blood tests. All results must be confirmed with a standard hemoglobin test (e.g. CBC or hemoglobinometer) before initiating clinical treatment.

14. Interfaces

14.1 REST API (FastAPI, `app/main.py`)

Endpoint	Purpose
`GET /`	Self-contained web UI (offline, no CDN)
`GET /health`	Liveness + **per-modality** model status (device, trained flag, classes)
`POST /analyze`	multipart: `image` + `metadata` (JSON string) + `with_gradcam` + `modality` (`eye`/`palm`) → full `TriageReport` (includes `gradcam_png_base64`, `heatmap_png_base64`, `disclaimer`)
`GET /api`	Machine-readable API index
`GET /docs`	Swagger UI served from **locally vendored assets** (works offline)

Metadata fields: `patient_id`, `age_months`, `sex`, `is_pregnant`, `trimester`, `symptoms[]` (auto lower-cased/normalized), `measured_hb` (optional; **overrides the image** for triage), `ifa_adherence`, `notes`. Validation errors → HTTP 422 with details.

14.2 Web UI (`app/static/`) — offline, self-contained

Inline CSS/JS (zero external dependencies — immune to CDN/network failures). Image upload with drag-and-drop + preview, modality dropdown (eye/palm), age/sex/pregnancy/trimester/ measured-Hb inputs, symptom chips (tap-to-toggle), Grad-CAM toggle, colour-coded result banner (🔴 High / 🟠 Medium / 🟢 Low), probability bars, quality-gate summary, inline Grad-CAM heatmap, warnings, per-modality model status badge in the header, and clinical disclaimer.

14.3 Interactive Dual-Modality Intake & Launcher (`run_pipeline.py`, `scripts/ui_selector.py`)

To ensure a seamless, crash-free clinical intake experience in real clinical environments, the system provides an interactive dual-modality workflow:

```
python run_pipeline.py
# or: make pipeline
# or: python -m scripts.analyze_cli
```

Key features of the interactive intake:

1. **Crash-Free Individual File Choosers:**
   Instead of unreliable auto-folder assignment (which risked misassigning palm images to the eyelid model), the intake provides two distinct buttons: `Select Eyelid Image` and `Select Palm Image`. On macOS, file selection is routed through native AppleScript (`osascript`) sheets (`choose file with prompt ...`). This completely bypasses the documented Tk 8.6 Cocoa `setAllowedFileTypes` `NSInvalidArgumentException` SIGABRT crash.
2. **Comprehensive Clinical Factors Form:**
   Frontline workers input:
   - **Age (years):** Automatically converted to months for pediatric WHO cut-offs.
   - **Biological Sex:** `female`, `male`, `other`, `unknown`.
   - **Pregnancy Status & Trimester:** `Yes`/`No` + Trimesters 1, 2, 3 (automatically applies pregnant WHO Hb bands).
   - **Measured Hemoglobin (optional):** Confirmatory Hb value if a blood test has already been run.
   - **Symptoms / Notes:** Free-text box parsed by the Clinical NLP engine.
3. **Interactive Terminal Fallback:**
   If running in a headless or terminal-only environment, `scripts/ui_selector.py` gracefully prompts for eyelid path, palm path, and clinical symptoms directly via interactive shell prompts.

14.4 Terminal CLI Analyzer (`scripts/analyze_cli.py`)

Supports both dual-modality and backward-compatible single-image execution:

```
# Dual-modality CLI invocation:
python -m scripts.analyze_cli \
  --eye data/sample/pale_pregnant.png \
  --palm "Palm/Anemic-260 (10).png" \
  --age-years 28 --sex female --pregnant --trimester 2 \
  --symptoms "severe fatigue, shortness of breath, dizziness"

# Legacy single-image invocation:
python -m scripts.analyze_cli --image data/sample/pale_pregnant.png --modality eye
```

- **Visual Dashboard Generation:**
  - `outputs/multimodal_triage_dashboard.png`: Master 2-row × 3-column figure displaying:
    - **Row 1:** Eyelid (Original Image · Preprocessed / Normalized ROI · Grad-CAM Overlay with predicted risk and confidence).
    - **Row 2:** Palm (Original Image · Preprocessed / Normalized ROI · Grad-CAM Overlay with predicted risk and confidence).
    - **Bottom Banner:** Highlighted Clinical Disclaimer banner.
  - `outputs/eyelid_3panel.png` & `outputs/palm_3panel.png`: Independent 3-panel figures for focused review.
  - `outputs/eyelid_heatmap.png` & `outputs/palm_heatmap.png`: Standalone high-contrast JET activation heatmaps.
  - `outputs/multimodal_triage_report.json`: Full machine-readable clinical JSON report.
- **Window Management:**
  The matplotlib figure is brought to the foreground on macOS using window lifting, topmost attributes, and AppleScript process activation (`osascript`). Passing `--open` automatically opens the master dashboard in macOS Preview.

- **Terminal Formatting:**
  Outputs an ANSI color-coded summary detailing Eyelid findings, Palm findings, Multimodal Fused Probability, NLP Symptom Analysis, Concordance Status, Action Code, Clinical Rationale, and concludes with the yellow **MANDATORY CLINICAL DISCLAIMER** box.

14.5 Batch processor (`scripts/batch_process.py`)

Folder of images → triage CSV (`--mode api` hits a running server; `--mode local` runs in-process). Per-image sidecar JSON supported. CSV columns: image, status, risk_class, p_anemic, confidence, model_trained, action_code, urgency, action, hb_band, flags, rule_fired, quality_passed, quality_reasons, processing_ms. Summary line reports REFER_NOW / high-urgency / rejected counts.

14.6 Make targets

`make install · samples · test · serve · manifest · train · train-eye · train-palm · evaluate · pipeline · cli IMG=… MODALITY=… · batch · export · notebook · clean`

15. Software Quality & Testing

55 automated tests, all passing (`pytest`):

Suite	Tests	Covers
`test_rules_engine.py`	15	Decision-table priorities, danger-sign override, measured-Hb override (severe/mild/normal), group-specific bands, boundary ages/Hb, pre-eclampsia flag, string inputs
`test_multimodal.py`	7	Clinical NLP danger signs extraction, NLP negation handling ("denies breathlessness"), NLP pregnancy/Hb extraction, blank/none handling, multimodal 70/30 weighted synthesis, eyelid rejection fallback to palm, mandatory clinical disclaimer verification
`test_api.py`	11	Health (per-modality), root UI, offline docs assets, analyze ok/rejected, red-flag → REFER_NOW, measured-Hb severe, bad metadata 422, palm modality, bad modality 422
`test_quality_gate.py`	6	Pass, blur reject, dark reject, no-tissue reject, Laplacian monotonicity, large-phone-photo resolution normalization
`test_preprocessing.py`	5	Decode roundtrip, bad bytes, auto-crop, tensor shape/dtype, full pass keys
`test_imageio.py`	5	PNG/JPEG bytes, corrupt bytes, file read, missing file, Pillow fallback
`test_clinical_config.py`	4	Config load, Hb band boundaries, pregnant/child cut-offs, danger-sign normalization
`test_cli.py`	2	Subprocess smoke (headless run saves figure), missing-image error

Additional QA: `pytest.ini` warnings policy; `py_compile` gate over all modules; every pipeline surface (API / CLI / batch) exercised end-to-end against the **trained checkpoints** and real dataset images during validation; ONNX export verified against onnxruntime (output shape (1,3)).

16. Engineering Issues Found & Fixed During the Build

Real defects discovered during validation and fixed (each verified after the fix):

1. **Train/serve preprocessing mismatch (the big one).** The first palm model scored 77.5% during training but, through the *serving* path, predicted "Mild/Moderate" for **100% of images** (degenerate). Root cause: training used albumentations transforms while serving used gray-world + CLAHE the model had never seen. Fix: training now uses the exact serving preprocessing with augmentation applied before it (§5.3); both models retrained; the serving-path evaluator now shows genuine class separation (palm 76.5%/69.7%).
2. **Quality gate rejected 100% of the real eye dataset.** Phone-photo-tuned thresholds (brightness ≥ 40, clip ≤ 35%) were incompatible with dark clinical crops on black backgrounds. Fix: measured the true

distributions across all 912/4,260 images and calibrated thresholds at the p5 percentile (§7.3).

3. **Palm blur threshold invalid.** Palms are smooth skin (median Laplacian variance 22.6) — the eye threshold (50) rejected every palm photo. Fix: modality-specific threshold (3).
4. **Biased evaluation sampling.** `evaluate` took the *first N* validation images; the manifest is label-sorted, so the sample was ~91% anemic and specificity was unmeasurable. Fix: random representative sampling with a fixed seed.
5. **HEIC/iPhone photos undecodable** (`cv2.imread` returns `None`). Fix: `app/imageio.py` with pillow-heif + EXIF transpose, used by every entry point.
6. **Large phone photos falsely flagged blurry.** A sharp 12-MP HEIC scored focus 5.8 (threshold 100). Fix: resolution normalization to 1024-px longest edge before metrics (§7.2) — same photo then scores 204.8.
7. **DataLoader worker crash** — `cv2` module stored on the dataset instance is unpicklable ("cannot pickle 'module' object"). Fix: imports moved inside `__getitem__`.
8. `torch.onnx.export` **failure** — torch 2.14 requires `onnxscript`. Fix: dependency added; export verified with onnxruntime.
9. **Swagger blank page** — FastAPI's `/docs` loads assets from a CDN; the build machine's HTTPS was timing out. Fix: vendored Swagger assets locally + a fully offline custom web UI at `/`.
10. **CLI pop-up window hidden** — the macOS matplotlib window opened behind the terminal. Fix: figure pre-draw, Tk lift/topmost, macOS process activation via `osascript`, plus `--open` (Preview) fallback.
11. **Enum serialization in the CLI report** — `RiskClass.mild_moderate` reprs instead of values. Fix: `model_dump(mode="json")`.
12. **Grad-CAM zero activation maps (flat blue).** The initial Grad-CAM implementation selected the last `Conv2d` layer in `model.modules()`, which resolved to `model.conv_head` in timm MobileNetV3. Because `conv_head` sits at/after the spatial pooling layer, its spatial dimensions collapsed to 1×1, producing identical gradients across spatial dimensions and an all-zero activation matrix (`min: 0.0, max: 0.0`). Fix: targeted `model.blocks[-1]` (the last 2D inverted residual block before pooling), restoring rich 224×224 spatial heatmaps with dynamic contrast (`min: 0.0, max: 1.0, std: 0.28`).
13. **Tkinter Cocoa** `setAllowedFileTypes` **SIGABRT crash on macOS.** Launching `tkinter.filedialog.askopenfilename` with compound semicolon patterns (e.g. `*.jpg;*.png`) crashed the Python runtime with a fatal `NSInvalidArgumentException: object cannot be nil` inside Apple's Tk Cocoa bridge. Fix: Replaced with native AppleScript execution (`osascript -e 'choose file with prompt ...'`), providing 100% crash-free native macOS Finder sheets.
14. **Folder auto-select image misattribution.** When scanning folders without strict filename conventions, heuristic auto-selection erroneously assigned palm images to the eyelid model (failing the conjunctiva quality gate). Fix: Removed ambiguous folder auto-selection in favor of two explicit individual file choosers (`Select Eyelid Image` and `Select Palm Image`), guaranteeing correct anatomical attribution.
15. **Zero-size crop assertion crashes in preprocessing.** Edge-case contours in low-contrast images caused OpenCV bounding box slices to collapse to 0-pixel widths/heights, causing `cv2.resize` assertion failures. Fix: Added defensive dimensions checks (`crop.size > 0 and crop.shape[0] > 0 and crop.shape[1] > 0`) in both `auto_crop_conjunctiva` and `_center_crop`.

17. Version History (Git)

Commit	Summary
`59f4346`	Initial Anemia-Screen pipeline: FastAPI + MobileNetV3 + WHO rules engine + quality gate + Grad-CAM + batch + tests
`e075e4e`	Self-contained web UI at `/` and offline Swagger `/docs` (fixes blank-page CDN issue)
`8d24679`	Terminal CLI analyzer: full summary + pop-up 3-panel Grad-CAM window
`368f083`	HEIC/iPhone photo support + resolution-independent blur gate
`8460d82`	Palm modality + training on real datasets: manifest builder, per-modality models, train/serve-consistent preprocessing, data-calibrated quality gate, honest evaluation
`Current`	Dual-modality screening pipeline (Eyelid 70% primary / Palm 30% secondary), Clinical NLP symptom analyzer, Grad-CAM spatial heatmap fix, native AppleScript intake UI, dual 3-panel dashboards, mandatory clinical disclaimer, 55 passing tests

18. Known Limitations & Honest Caveats

1. **Not a diagnostic device.** Screening/triage decision support only; every abnormal result requires confirmatory measured Hb. Not cleared for clinical use (CDSCO Medical Device Rules apply to diagnostic software).
2. **Validation, not held-out test.** Metrics are from the validation split (best checkpoint selected on it). A final untouched test set and external validation are future work.
3. **Class imbalance & small severe class.** Only 48 severe eye images; severe-class recall is weak (2/10 in the sampled evaluation) — the safety net is that severe *presentations* (danger signs, measured Hb) override the image entirely.
4. **Dataset heterogeneity.** Eye training mixes three countries/cameras; subgroup performance by skin tone, lighting and phone was **not** measured — the main failure mode per the spec, flagged for the pilot phase.
5. **Palm labels are binary proxies.** Anemic palms map to Mild/Moderate without Hb values, so the palm model's 3-class output is effectively anemic/non-anemic.
6. **No altitude/smoking Hb adjustment** (WHO allows both; out of scope).
7. **Overfitting signs** (train loss ≪ val loss from ~epoch 3 palm / epoch 8 eye); best-checkpoint selection mitigates but more data/regularization would help.
8. **Single-modality per report [RESOLVED in v1.1.0].** The pipeline now supports integrated dual-modality evaluation with 70% Eyelid / 30% Palm probability fusion, cross-site concordance auditing, and graceful single-site fallback.

19. Roadmap

- ☑ **Fused two-site analysis** (eyelid primary 70% + palm corroboration 30%, agreement/disagreement auditing) — *Completed in v1.1.0*
- ☑ **Clinical NLP symptom intake** with negation handling and danger-sign escalation — *Completed in v1.1.0*
- ☑ **Dual 3-panel visual dashboard & standalone heatmaps** — *Completed in v1.1.0*
- ☐ Held-out test split + subgroup bias evaluation (skin tone / lighting / phone)
- ☐ WHO-2024 trimester-specific Hb cut-offs (config-ready in `clinical_config.yaml`)
- ☐ Sensitivity-threshold tuning with ROC analysis on a larger validation cohort
- ☐ ONNX/TFLite on-device inference + offline-first sync (ASHA field app)
- ☐ Altitude/smoking adjustments; multi-signal model (conjunctiva + nail bed + symptoms)

- ☐ RCH portal / ABDM health-ID integration; CDSCO regulatory pathway; consent flow per DPDP 2023
- ☐ Prospective clinical validation on a locally collected, consented, hemoglobinometer-confirmed cohort

20. Runbook (How to Run Everything)

```
cd "/Users/allen2775/Desktop/cline hackathon/anemia-screen"
conda activate anemia          # one-time env setup: conda env create -f environment.yml

# ── 1. Interactive Dual-Modality Screening (Recommended) ──
python run_pipeline.py         # Opens native dialog to select Eyelid & Palm individually,
                               # collects clinical factors, analyzes symptoms, and shows
                                   dashboard
# or via Make:
make pipeline

# ── 2. Direct CLI Dual-Modality Invocation ──
python -m scripts.analyze_cli \
  --eye data/sample/pale_pregnant.png \
  --palm "Palm/Anemic-260 (10).png" \
  --age-years 28 --sex female --pregnant --trimester 2 \
  --symptoms "severe fatigue, shortness of breath, dizziness"

# ── 3. Single-Image Analyzer (Legacy Mode) ──
python -m scripts.analyze_cli --image data/sample/pale_pregnant.png --modality eye

# ── 4. Web Service & API ──
uvicorn app.main:app --reload   # → http://127.0.0.1:8000/ (UI) · /docs (Swagger)

# ── 5. Batch CSV Processing ──
python -m scripts.batch_process --mode local --input data/sample --out outputs/triage.csv

# ── 6. Quality Assurance & Testing ──
pytest                         # 55 passing automated tests
python -m ml.export_onnx       # checkpoint → ONNX (onnxruntime check)

# ── 7. Model Retraining & Evaluation (Optional) ──
python -m ml.build_manifest
python -m ml.train --modality eye  --epochs 12 --batch-size 32
python -m ml.train --modality palm --epochs 8  --batch-size 64
python -m ml.evaluate --modality eye  --max-images 150
python -m ml.evaluate --modality palm --max-images 200
```

21. References

- NFHS-5 (2019–21), IIPS & MoHFW — anemia prevalence figures
- Anemia Mukt Bharat operational guidelines, MoHFW (test-and-treat, IFA schedules)
- WHO: *Haemoglobin concentrations for the diagnosis of anaemia and assessment of severity* (2011); WHO 2024 guideline on haemoglobin cut-offs (config-ready)
- WHO: *Guideline on daily iron and folic acid supplementation in pregnant women*
- WHO: *Ethics and governance of artificial intelligence for health* (2021)
- ICMR: *Ethical guidelines for AI in biomedical research and healthcare* (2023)
- Digital Personal Data Protection Act, 2023; CDSCO Medical Device Rules, 2017
- Eyes-defy-anemia dataset (India/Italy); CP-AnemiC dataset (Ghana); palmar images anemia dataset ("Detecting Anemia Based on Palm Images" family)
- Project docs: `README.md`, `docs/DATASETS.md`, `docs/CLINICAL_AND_ETHICS.md`

Report generated 10 October 2026. All metrics in §10–11 are reproducible via the runbook commands in §20 on this machine (seed 42, patient-wise splits, MPS device).